

Algorithm Complexity Analysis

UFUG 2160 Tutorial 12

Bohan Hu

April 29, 2025

Ph.D. Cadidate, Microelectronics Thrust, Function Hub
The Hong Kong University of Science and Technology (Guangzhou)



Project 2 Guide

Background

- ▶ RSA is a public-key cryptosystem widely used for secure data transmission.
- ▶ It is based on the mathematical properties of prime numbers and modular arithmetic.
- ▶ Developed in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman.

Asymmetric Encryption

- ▶ Two keys work together: *public key* and *private key*
- ▶ Anyone having *public key* can encrypt message
- ▶ Only the ones who have *private key* can decrypt the encrypted message

Key Generation

1. Choose two distinct large prime numbers p and q .
2. Compute $n = p \cdot q$. n is the modulus for both the public and private keys.
3. Compute $\phi(n) = (p - 1)(q - 1)$, where ϕ is Euler's totient function.
4. Choose an integer e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
5. Compute the private key d such that $d \cdot e \equiv 1 \pmod{\phi(n)}$.

Encryption: Using Public Key

- ▶ Public key: (e, n)
- ▶ Convert the plaintext message M into a number M such that $0 \leq M < n$.
- ▶ Compute the ciphertext C using the formula:

$$C = M^e \mod n$$

- ▶ Transmit C to the recipient.

Decryption: Using Private Key

- ▶ Private key: (d, n)
- ▶ The recipient receives the ciphertext C .
- ▶ Compute the original message M using the formula:

$$M = C^d \mod n$$

- ▶ Since $d \cdot e \equiv 1 \pmod{\phi(n)}$, the decryption retrieves the original message.

Example of RSA

Key Generation

- ▶ Choose $p = 61, q = 53$.

Encryption and Decryption

Example of RSA

Key Generation

- ▶ Choose $p = 61, q = 53$.
- ▶ Compute $n = p \cdot q = 61 \cdot 53 = 3233$.

Encryption and Decryption

Example of RSA

Key Generation

- ▶ Choose $p = 61, q = 53$.
- ▶ Compute $n = p \cdot q = 61 \cdot 53 = 3233$.
- ▶ Compute $\phi(n) = (p - 1)(q - 1) = 60 \cdot 52 = 3120$.

Encryption and Decryption

Example of RSA

Key Generation

- ▶ Choose $p = 61, q = 53$.
- ▶ Compute $n = p \cdot q = 61 \cdot 53 = 3233$.
- ▶ Compute $\phi(n) = (p - 1)(q - 1) = 60 \cdot 52 = 3120$.
- ▶ Choose $e = 17$ (common choice, satisfies $\gcd(e, \phi(n)) = 1$).

Encryption and Decryption

Example of RSA

Key Generation

- ▶ Choose $p = 61, q = 53$.
- ▶ Compute $n = p \cdot q = 61 \cdot 53 = 3233$.
- ▶ Compute $\phi(n) = (p - 1)(q - 1) = 60 \cdot 52 = 3120$.
- ▶ Choose $e = 17$ (common choice, satisfies $\gcd(e, \phi(n)) = 1$).
- ▶ Compute d such that $d \cdot e \equiv 1 \pmod{3120}$, resulting in $d = 2753$.

Encryption and Decryption

Example of RSA

Key Generation

- ▶ Choose $p = 61, q = 53$.
- ▶ Compute $n = p \cdot q = 61 \cdot 53 = 3233$.
- ▶ Compute $\phi(n) = (p - 1)(q - 1) = 60 \cdot 52 = 3120$.
- ▶ Choose $e = 17$ (common choice, satisfies $\gcd(e, \phi(n)) = 1$).
- ▶ Compute d such that $d \cdot e \equiv 1 \pmod{3120}$, resulting in $d = 2753$.
- ▶ Public key: $(e, n) = (17, 3233)$; Private key: $(d, n) = (2753, 3233)$.

Encryption and Decryption

Example of RSA

Key Generation

- ▶ Choose $p = 61, q = 53$.
- ▶ Compute $n = p \cdot q = 61 \cdot 53 = 3233$.
- ▶ Compute $\phi(n) = (p - 1)(q - 1) = 60 \cdot 52 = 3120$.
- ▶ Choose $e = 17$ (common choice, satisfies $\gcd(e, \phi(n)) = 1$).
- ▶ Compute d such that $d \cdot e \equiv 1 \pmod{3120}$, resulting in $d = 2753$.
- ▶ Public key: $(e, n) = (17, 3233)$; Private key: $(d, n) = (2753, 3233)$.

Encryption and Decryption

- ▶ Encrypt $M = 65$:

$$C = 65^{17} \pmod{3233} = 2790$$

Example of RSA

Key Generation

- ▶ Choose $p = 61, q = 53$.
- ▶ Compute $n = p \cdot q = 61 \cdot 53 = 3233$.
- ▶ Compute $\phi(n) = (p - 1)(q - 1) = 60 \cdot 52 = 3120$.
- ▶ Choose $e = 17$ (common choice, satisfies $\gcd(e, \phi(n)) = 1$).
- ▶ Compute d such that $d \cdot e \equiv 1 \pmod{3120}$, resulting in $d = 2753$.
- ▶ Public key: $(e, n) = (17, 3233)$; Private key: $(d, n) = (2753, 3233)$.

Encryption and Decryption

- ▶ Encrypt $M = 65$:

$$C = 65^{17} \pmod{3233} = 2790$$

- ▶ Decrypt $C = 2790$:

$$M = 2790^{2753} \pmod{3233} = 65$$

Program Execution Flow

- ▶ Initialization (Sender): Ask receiver for a **public key**
- ▶ Initialization (Receiver)
 1. `private_key = gen_private_key()`
 2. `public_key = gen_public_key(private_key)`
 3. Send `public_key` to sender
- ▶ Encryption (Sender)
 1. `message_bytes = message.encode("utf-8")`
 2. `ciphertext = encrypt(message_bytes, public_key)`
- ▶ Encryption output & Decryption input: `cyphertext`, `public_key`
- ▶ Decryption
 1. `message_bytes = decrypt(ciphertext, private_key)`
 2. `message = message_bytes.decode("utf-8")`

Note: Processing Strings

- ▶ Before encryption, encode the strings with specific encoding scheme.
e.g., `encoded_string = string.encode('utf-8')`
- ▶ Treat them as normal arrays and do encryption
- ▶ After decryption, decode the strings with the **same** encoding scheme.
e.g., `decoded_string = decrypted.decode('utf-8')`

Algorithm Complexity Analysis

Big- O , Big- Θ and Big- Ω

For $f(n)$:

- ▶ $\Omega(g)$: $Ag(n) \leq f(n)$ for every integer $n \geq a$
- ▶ $O(g)$: $0 \leq f(n) \leq Bg(n)$ for every integer $n \geq a$
- ▶ $\Theta(g)$: $Ag(n) \leq f(n) \leq Bg(n)$ for every integer $n \geq a$

$g(n) \geq 0$ for every integer $n \geq r$.

Basic Idea: Determine A , B and a

Example: Finding big- Ω by Inequality Construction

Show that $16n^4 + 11n^3 + 8$ is $\Omega(n^4)$. Remember: $\Omega(g)$: $Ag(n) \leq f(n)$

1. $g(n) = n^4$. Let $A = 16$, then we have $16n^4 \leq 16n^4 + 11n^3 + 8$ for every integer $n \geq a$
2. Try to find an existing a s.t. $0 \leq 11n^3 + 8$. Obviously $a = 1$ satisfies this condition.
3. Let $A = 16, a = 1$, proved.

Finding big- O / big- Ω for Polynomials with Negative Coeff

How different?

1. $n^4 + 5n + 8$ is $\Omega(n^4)$?
2. $n^4 - 5n - 8$ is $\Omega(n^4)$?

For (1), we can always say $n^4 + 5n + 8 \leq n^4 + 5n^4 + 8n^4$ for $n \geq 1$.

For (2), it is uncertain. We should find another way to construct the inequality.

Let m be a nonnegative integer, let $P(n)$ be a polynomial of degree m , and suppose the coefficient a_m of n^m is positive.

To find big- Ω for $P(n)$: Let $A = \frac{1}{2}a_m$, and let a be the number obtained as follows:

1. Find the sum of the absolute values of all the coefficients of $P(n)$ except for a_m .
2. Multiply the result of step 1 by $\frac{2}{a_m}$.
3. Let a be the larger of the number 1 and the result of step 2.

Show that $An^m \leq P(n)$ for every integer $n \geq a$.

Example: Polynomial Function with Some Negative Coefficients

- ▶ The coefficient of the highest power, n^4 , is 1.
- ▶ The sum of the absolute values of the other coefficients is:

$$|-5| + |-8| = 5 + 8 = 13.$$

- ▶ Calculate a :

$$a = \frac{2}{1}(5 + 8) = 26, \quad \text{which is greater than 1.}$$

- ▶ Require $n \geq a$

$$n \geq \frac{2}{1} (|-5| + |-8|) = 26.$$

Example: Polynomial Function with Some Negative Coefficients

- ▶ Multiply both sides by $\frac{1}{2}n^3$

$$\frac{1}{2}n^4 \geq (|-5| + |-8|)n^3.$$

Simplify:

$$\frac{1}{2}n^4 \geq 5n^3 + 8n^3 \quad (\text{because } n \geq 1 \text{ and } 5n^3 \geq 5n \text{ and } 8n^3 \geq 8).$$

Hence:

$$\frac{1}{2}n^4 \geq 5n + 8.$$

- ▶ Subtract the right-hand side, and add $\frac{1}{2}n^4$ to both sides:

$$n^4 - 5n - 8 \geq \frac{1}{2}n^4 \quad \text{for every integer } n \geq a.$$

Summary

Case	With $a_m < 0$	All $a_m \geq 0$
$\Omega(g(n))$	Generic Method	Omit Lower-Order Terms
$O(g(n))$	Negate the Lower-Order Terms	Raise the Order of Lower-Order Terms

1. $n^4 + 5n + 8$

2. $n^4 - 5n - 8$

Common Algorithm Analysis Techniques

Analyzing Loops

Loop in Algorithm

- ▶ Perfectly nested loops: Compute the total number of iterations - **for** loops: loop bounds; **while** loops: termination condition
- ▶ Compute the actual number of executed elementary operations
 - addition, subtraction, multiplication, division
 - comparison

Example: Bubble Sort

Algorithm 1 Bubble Sort Algorithm

Require: An array of length n , `array`

Ensure: A sorted array `array`

```
1: for  $i = 0 \rightarrow n - 1$  do  
2:   for  $j = 0 \rightarrow n - 1 - i$  do  
3:     if array[j] > array[j+1] then  
4:       swap(array[j], array[j+1])  
5:     end if  
6:   end for  
7: end for
```

Example: Textbook Exercise

```
for  $i := 1$  to  $n - 1$   
   $p := 1$   
   $q := 1$   
  for  $j := i + 1$  to  $n$   
     $p := p \cdot c[j]$   
     $q := q \cdot (c[j])^2$   
  next  $j$   
   $r := p + q$   
next  $i$ 
```

Recursion

General Steps

1. Write down the relationship between current problem of size n and particular sub-problem of size n' , i.e., $T_n = f(T_{n'})$
2. Generalize to expressions like $T_n = f(g(T_n))$
3. Let $g(T_n) = 1$, find the baseline condition for recursion

Example: Quick Sort - Best-Case

- Under the **best-case scenario**, the array is evenly divided into two sub-arrays at each **PARTITION** step, leading to the recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + Cn,$$

where C is a constant representing the time for the partition step.

Example: Quick Sort - Best-Case

- Under the **best-case scenario**, the array is evenly divided into two sub-arrays at each PARTITION step, leading to the recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + Cn,$$

where C is a constant representing the time for the partition step.

- Expanding the recurrence step by step:

$$T(n) = 2T\left(\frac{n}{2}\right) + Cn,$$

$$T(n) = 2\left[2T\left(\frac{n}{4}\right) + C\frac{n}{2}\right] + Cn,$$

$$T(n) = 4T\left(\frac{n}{4}\right) + Cn + Cn,$$

$$T(n) = 8T\left(\frac{n}{8}\right) + Cn + Cn + Cn.$$

Example: Quick Sort - Best-Case (Cont'd)

- ▶ Generalizing after k steps:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kCn.$$

Example: Quick Sort - Best-Case (Cont'd)

- ▶ Generalizing after k steps:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kCn.$$

- ▶ The recursion stops when the subproblem size becomes 1, i.e.,:

$$\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n.$$

Example: Quick Sort - Best-Case (Cont'd)

- ▶ Generalizing after k steps:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kCn.$$

- ▶ The recursion stops when the subproblem size becomes 1, i.e.,:

$$\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n.$$

- ▶ Substitute $k = \log_2 n$ and the base case $T(1) = 1$ into the generalized recurrence:

$$T(n) = 2^{\log_2 n} T(1) + (\log_2 n)Cn.$$

Since $2^{\log_2 n} = n$:

$$T(n) = n \cdot T(1) + Cn \log_2 n.$$

Example: Quick Sort - Best-Case (Cont'd)

- ▶ Generalizing after k steps:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kCn.$$

- ▶ The recursion stops when the subproblem size becomes 1, i.e.,:

$$\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n.$$

- ▶ Substitute $k = \log_2 n$ and the base case $T(1) = 1$ into the generalized recurrence:

$$T(n) = 2^{\log_2 n} T(1) + (\log_2 n)Cn.$$

Since $2^{\log_2 n} = n$:

$$T(n) = n \cdot T(1) + Cn \log_2 n.$$

- ▶ Using $T(1) = 1$, we have:

$$T(n) = n + Cn \log_2 n.$$

Quick Sort - Worst-Case

- ▶ In the **worst-case scenario**, the pivot at each **PARTITION** step divides the array into two highly imbalanced sub-arrays, with one sub-array containing $n - 1$ elements and the other containing 0 elements. This leads to the recurrence relation:

$$T(n) = T(n - 1) + Cn,$$

Quick Sort - Worst-Case

- ▶ In the **worst-case scenario**, the pivot at each **PARTITION** step divides the array into two highly imbalanced sub-arrays, with one sub-array containing $n - 1$ elements and the other containing 0 elements. This leads to the recurrence relation:

$$T(n) = T(n - 1) + Cn,$$

- ▶ Expand the recurrence step by step:

$$T(n) = T(n - 1) + Cn,$$

$$T(n) = [T(n - 2) + C(n - 1)] + Cn,$$

$$T(n) = [[T(n - 3) + C(n - 2)] + C(n - 1)] + Cn.$$

Quick Sort - Worst-Case

- ▶ In the **worst-case scenario**, the pivot at each **PARTITION** step divides the array into two highly imbalanced sub-arrays, with one sub-array containing $n - 1$ elements and the other containing 0 elements. This leads to the recurrence relation:

$$T(n) = T(n - 1) + Cn,$$

- ▶ Expand the recurrence step by step:

$$T(n) = T(n - 1) + Cn,$$

$$T(n) = [T(n - 2) + C(n - 1)] + Cn,$$

$$T(n) = [[T(n - 3) + C(n - 2)] + C(n - 1)] + Cn.$$

- ▶ After k expansions:

$$T(n) = T(n - k) + C \sum_{i=0}^{k-1} (n - i).$$

Quick Sort - Worst-Case (Cont'd)

- ▶ The recursion stops when the subproblem size becomes 1, i.e., $n - k = 1$, which implies:

$$k = n - 1.$$

Quick Sort - Worst-Case (Cont'd)

- ▶ The recursion stops when the subproblem size becomes 1, i.e., $n - k = 1$, which implies:

$$k = n - 1.$$

- ▶ Substitute $k = n - 1$ and $T(1) = 1$ into the generalized recurrence:

$$T(n) = T(1) + C \sum_{i=0}^{n-2} (n - i).$$

Quick Sort - Worst-Case (Cont'd)

- ▶ The recursion stops when the subproblem size becomes 1, i.e., $n - k = 1$, which implies:

$$k = n - 1.$$

- ▶ Substitute $k = n - 1$ and $T(1) = 1$ into the generalized recurrence:

$$T(n) = T(1) + C \sum_{i=0}^{n-2} (n - i).$$

- ▶ Simplify the summation:

$$\sum_{i=0}^{n-2} (n - i) = n + (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n + 1)}{2}.$$

Quick Sort - Worst-Case (Cont'd)

- ▶ The recursion stops when the subproblem size becomes 1, i.e., $n - k = 1$, which implies:

$$k = n - 1.$$

- ▶ Substitute $k = n - 1$ and $T(1) = 1$ into the generalized recurrence:

$$T(n) = T(1) + C \sum_{i=0}^{n-2} (n - i).$$

- ▶ Simplify the summation:

$$\sum_{i=0}^{n-2} (n - i) = n + (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n + 1)}{2}.$$

- ▶ Thus:

$$T(n) = 1 + C \frac{n(n + 1)}{2}.$$

Master Theorem for Divide-and-Conquer Recurrences

The **Master Theorem** provides a way to determine the time complexity of divide-and-conquer algorithms expressed by the recurrence relation:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

where:

- ▶ $a \geq 1$: Number of subproblems,
- ▶ $b > 1$: Factor by which the problem size is divided,
- ▶ $f(n)$: Cost of dividing and combining the subproblems.

Master Theorem

Case Analysis:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

$$p = \log_b a \quad (\text{critical exponent}).$$

1. If $f(n) = O(n^{p-\epsilon})$ for some $\epsilon > 0$, then:

$$T(n) = \Theta(n^p).$$

2. If $f(n) = \Theta(n^p)$, then:

$$T(n) = \Theta(n^p \log n).$$

3. If $f(n) = \Omega(n^{p+\epsilon})$ for some $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some $c < 1$, then:

$$T(n) = \Theta(f(n)).$$

Applications of Master Theorem

Example 1:

$$T(n) = 2T\left(\frac{n}{2}\right) + n.$$

Here:

- ▶ $a = 2, b = 2, f(n) = n,$
- ▶ $p = \log_2 2 = 1.$

Since $f(n) = \Theta(n^p)$, this is **Case 2**, and the solution is:

$$T(n) = \Theta(n \log n).$$

Applications of Master Theorem

Example 2:

$$T(n) = 3T\left(\frac{n}{4}\right) + n^2.$$

Here:

- ▶ $a = 3, b = 4, f(n) = n^2,$
- ▶ $p = \log_4 3 \approx 0.79.$

Since $f(n) = \Omega(n^{p+\epsilon})$ for $\epsilon > 0$, this is **Case 3**, and the solution is:

$$T(n) = \Theta(n^2).$$

Thank you!